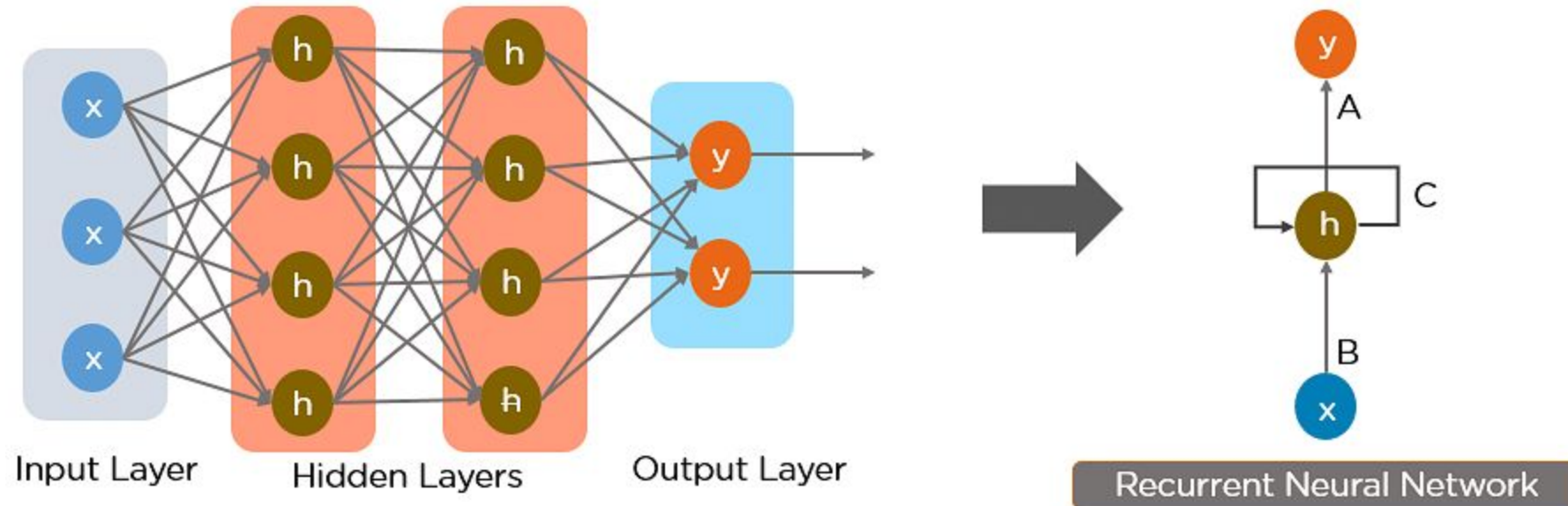
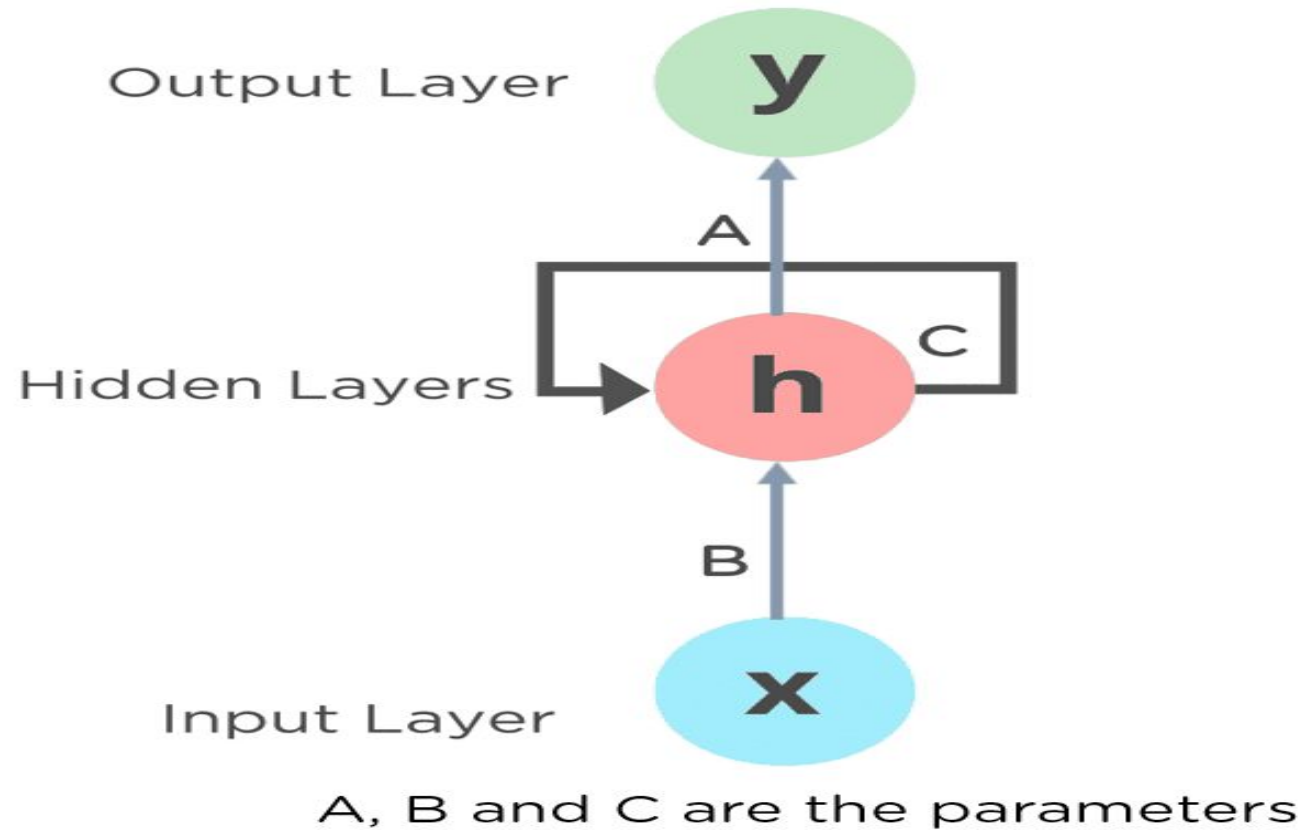


What Is a Recurrent Neural Network (RNN)?

- RNN works on the principle of saving the output of a particular layer and feeding this back to the input in order to predict the output of the layer.
- Below is how you can convert a Feed-Forward Neural Network into a Recurrent Neural Network

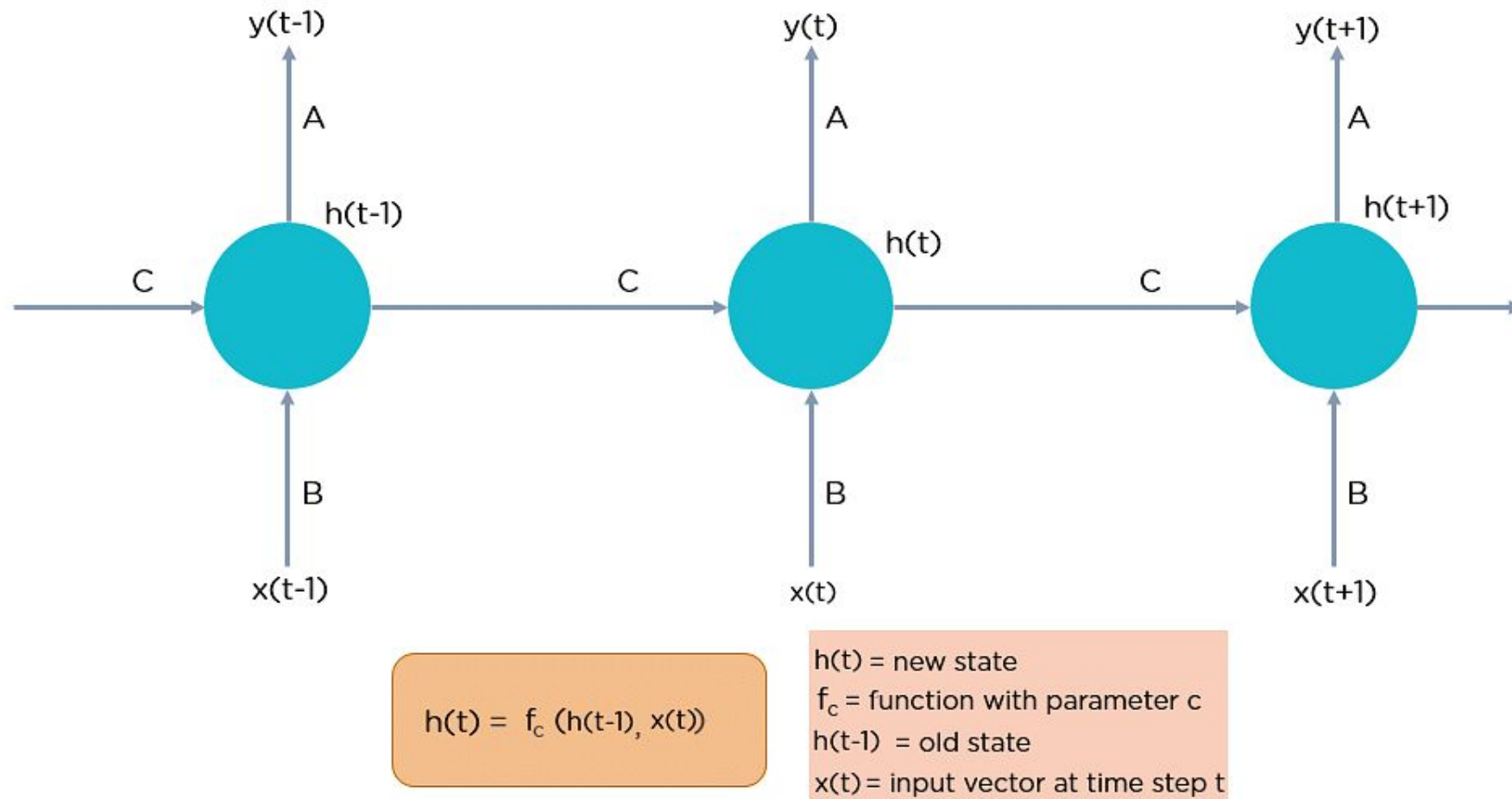


- The nodes in different layers of the neural network are compressed to form a single layer of recurrent neural networks.
- A, B, and C are the parameters of the network.



Fully connected Recurrent Neural Network

- Here, “x” is the input layer, “h” is the hidden layer, and “y” is the output layer.
- A, B, and C are the network parameters used to improve the output of the model.
- At any given time t, the current input is a combination of input at x(t) and x(t-1). The output at any given time is fetched back to the network to improve on the output.

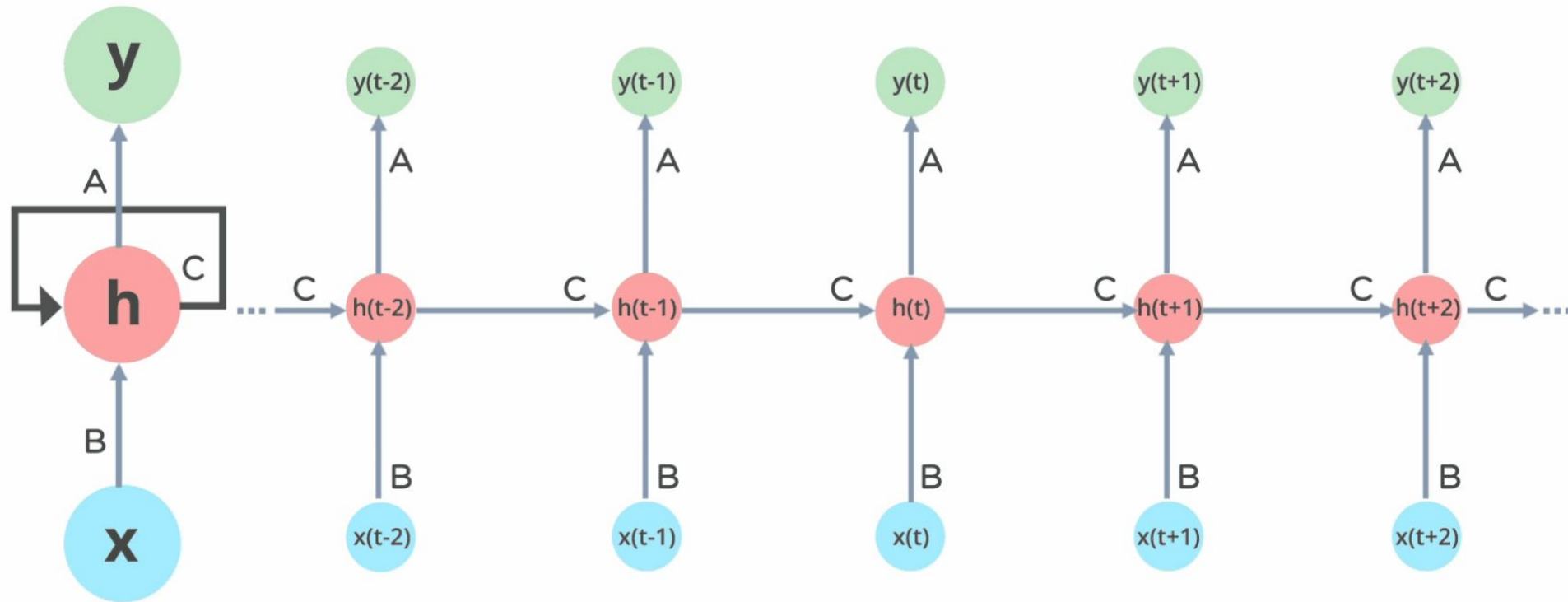


Why Recurrent Neural Networks?

- RNN were created because there were a few issues in the feed-forward neural network:
 - Cannot handle sequential data
 - Considers only the current input
 - Cannot memorize previous inputs
- The solution to these issues is the RNN.
- An RNN can handle sequential data, accepting the current input data, and previously received inputs.
- RNNs can memorize previous inputs due to their internal memory.

How Does Recurrent Neural Networks Work?

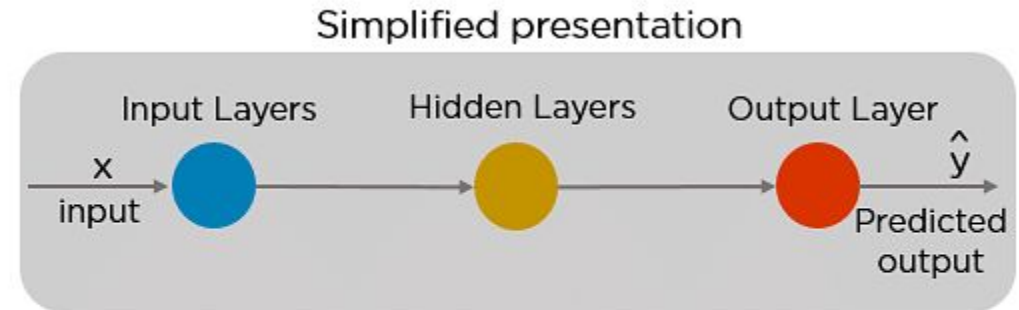
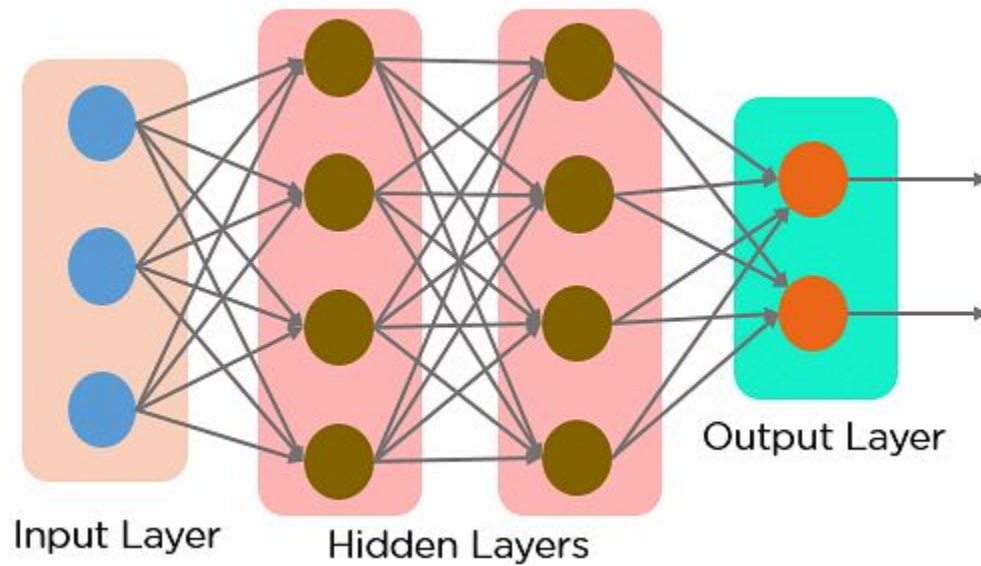
- In Recurrent Neural networks, the information cycles through a loop to the middle hidden layer.



- The input layer 'x' takes in the input to the neural network and processes it and passes it onto the middle layer.
- The middle layer 'h' can consist of multiple hidden layers, each with its own activation functions and weights and biases.
- If you have a neural network where the various parameters of different hidden layers are not affected by the previous layer, ie: the neural network does not have memory, then you can use a recurrent neural network.
- The Recurrent Neural Network will standardize the different activation functions and weights and biases so that each hidden layer has the same parameters.
- Then, instead of creating multiple hidden layers, it will create one and loop over it as many times as required.

Feed-Forward Neural Networks vs Recurrent Neural Networks

- A feed-forward neural network allows information to flow only in the forward direction, from the input nodes, through the hidden layers, and to the output nodes.
- There are no cycles or loops in the network.



Feed-forward neural networks

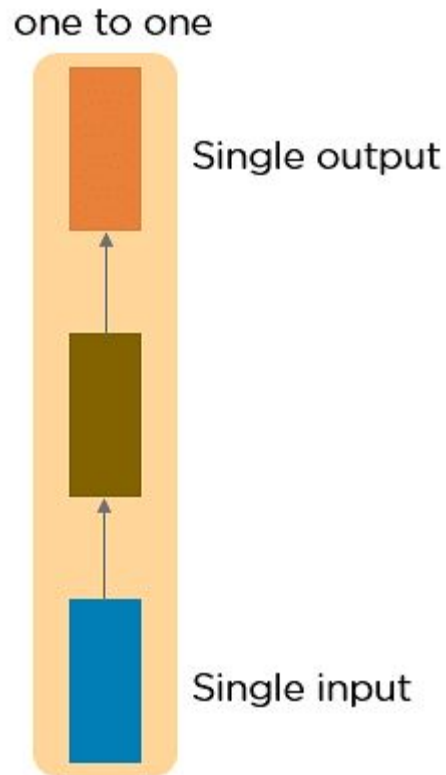
- In a feed-forward neural network, the decisions are based on the current input.
- It doesn't memorize the past data, and there's no future scope.
- Feed-forward neural networks are used in general regression and classification problems.

Types of Recurrent Neural Networks

- There are four types of Recurrent Neural Networks:
 1. One to One
 2. One to Many
 3. Many to One
 4. Many to Many

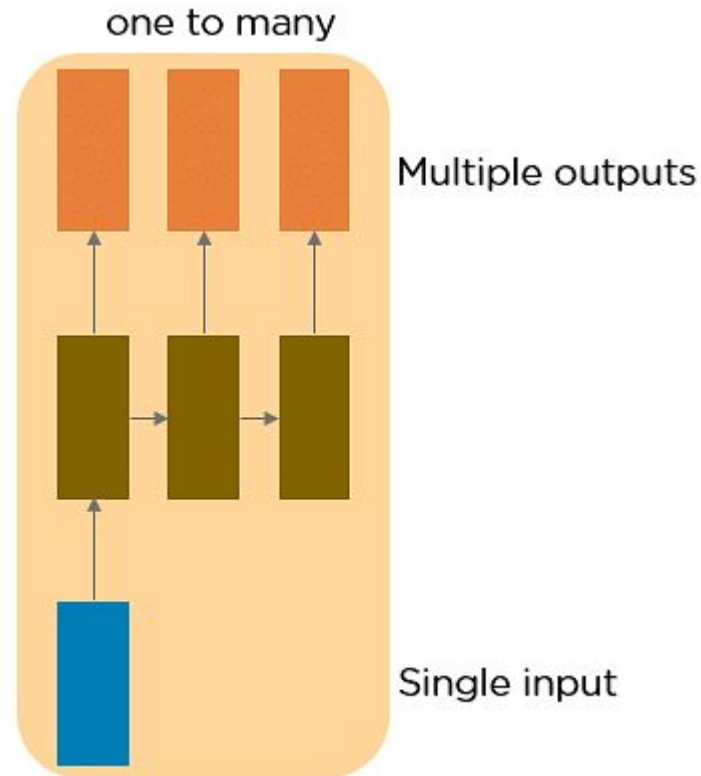
One to One RNN

- This type of neural network is known as the Vanilla Neural Network.
- It's used for general machine learning problems, which has a single input and a single output.



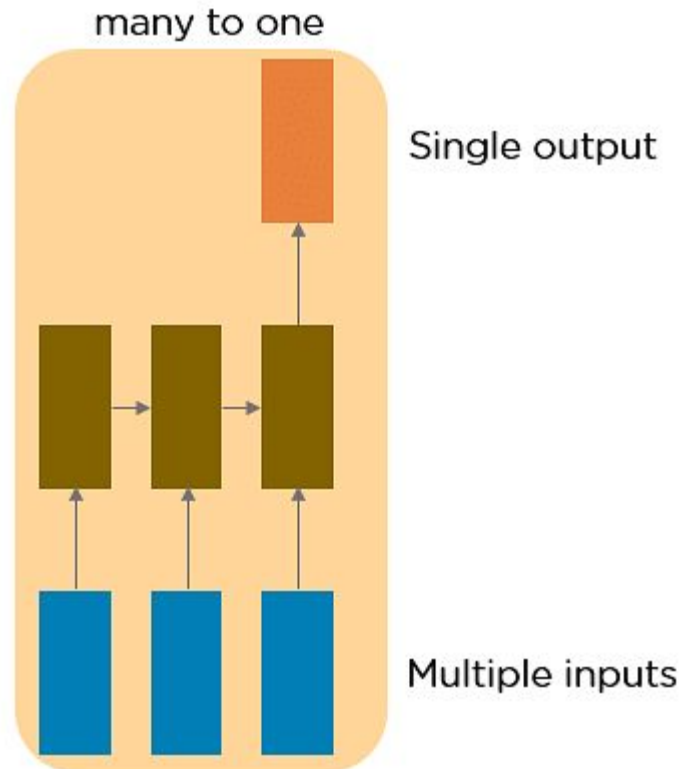
One to Many RNN

- This type of neural network has a single input and multiple outputs.
- An example of this is the image caption.



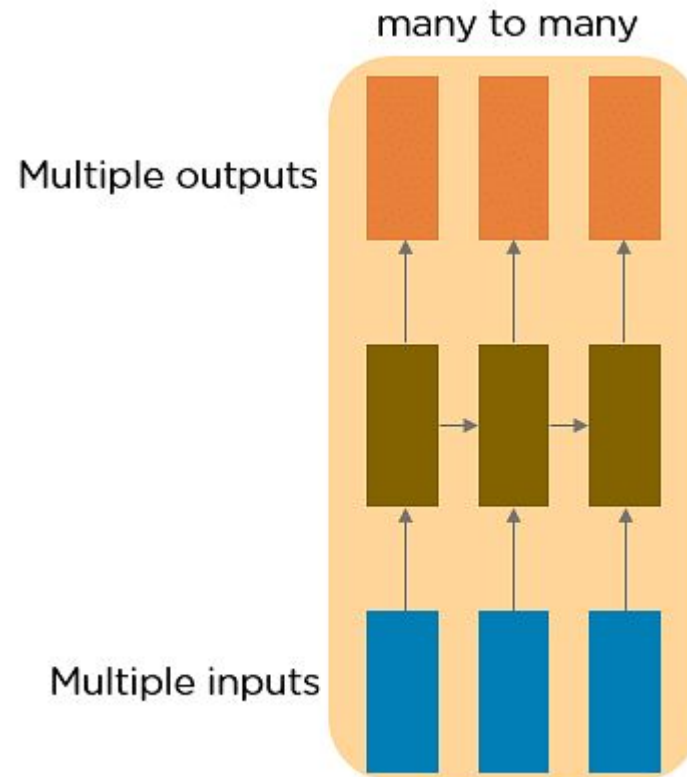
Many to One RNN

- This RNN takes a sequence of inputs and generates a single output.
- Sentiment analysis is a good example of this kind of network where a given sentence can be classified as expressing positive or negative sentiments.



Many to Many RNN

- This RNN takes a sequence of inputs and generates a sequence of outputs.
- Machine translation is one of the examples.

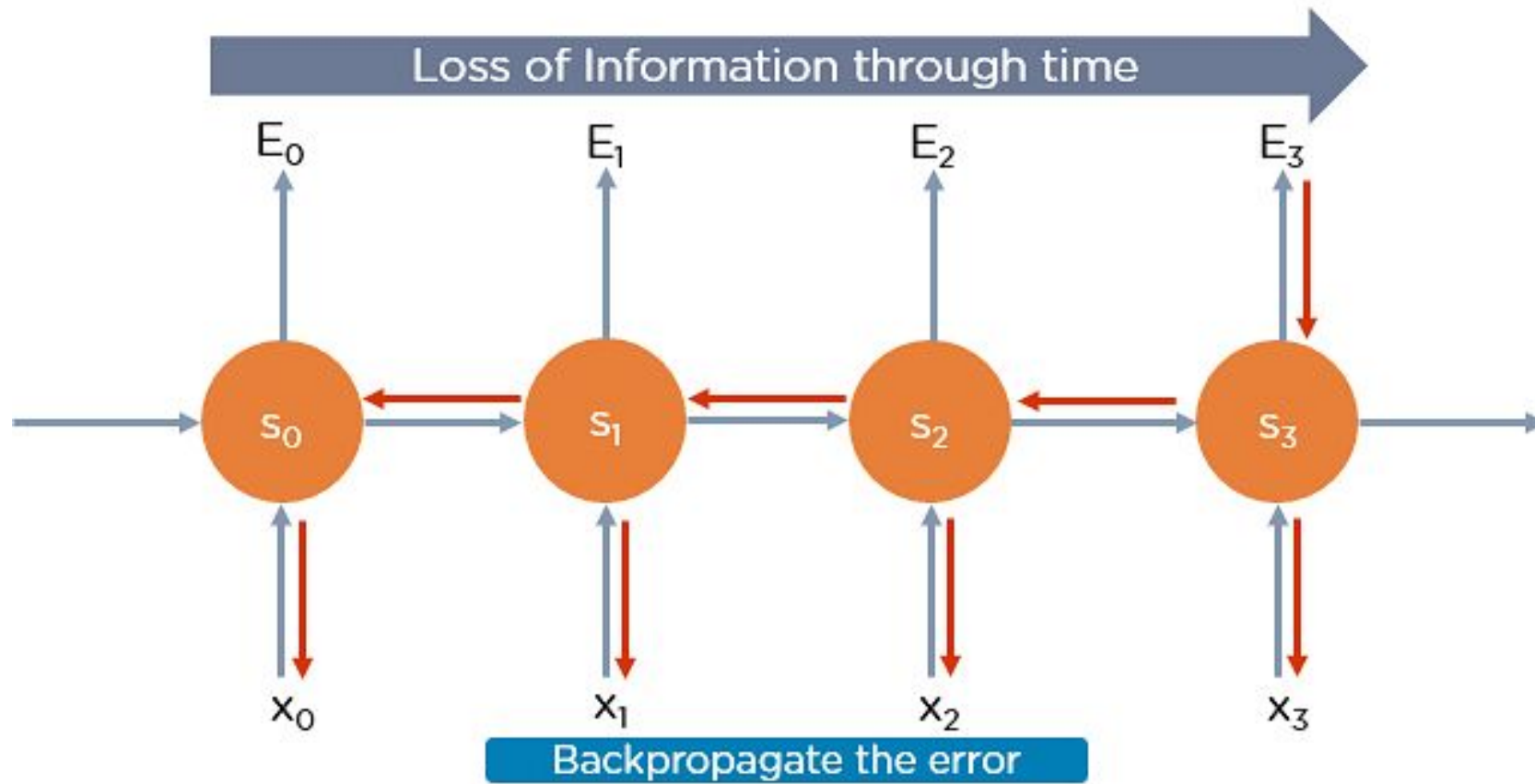


Two Issues of Standard RNNs

Vanishing Gradient Problem

- Recurrent Neural Networks enable you to model time-dependent and sequential data problems, such as stock market prediction, machine translation, and text generation.
- You will find, however, RNN is hard to train because of the gradient problem.
- RNNs suffer from the problem of vanishing gradients.
- The gradients carry information used in the RNN, and when the gradient becomes too small, the parameter updates become insignificant.
- This makes the learning of long data sequences difficult.

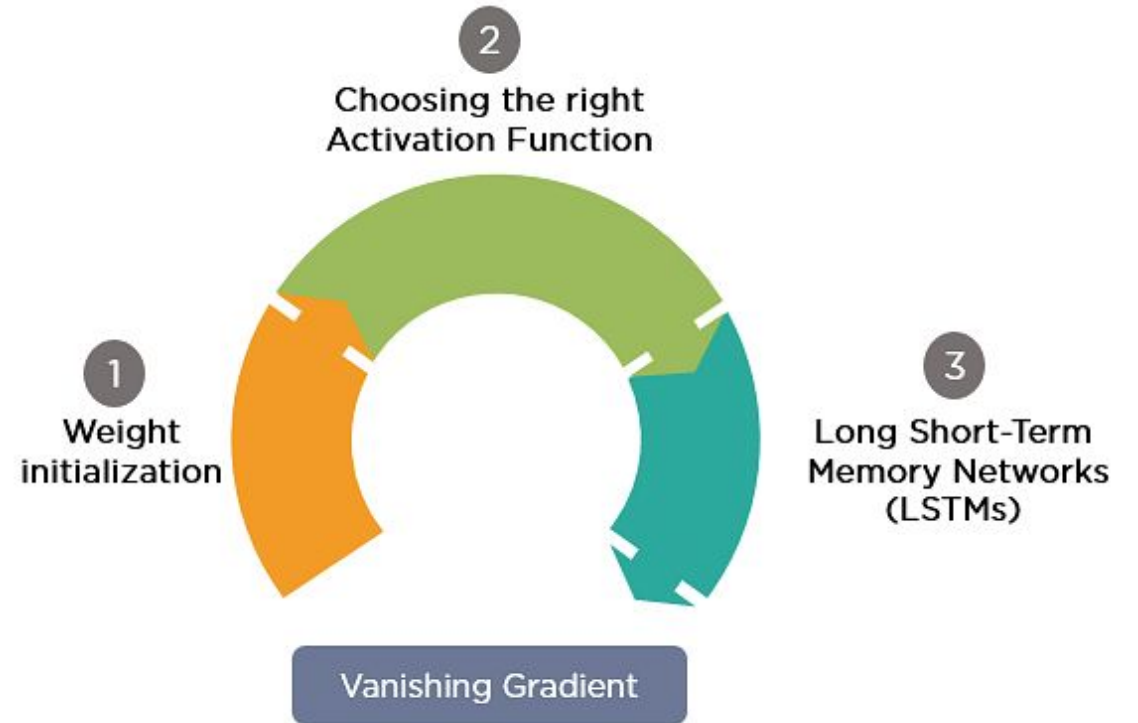
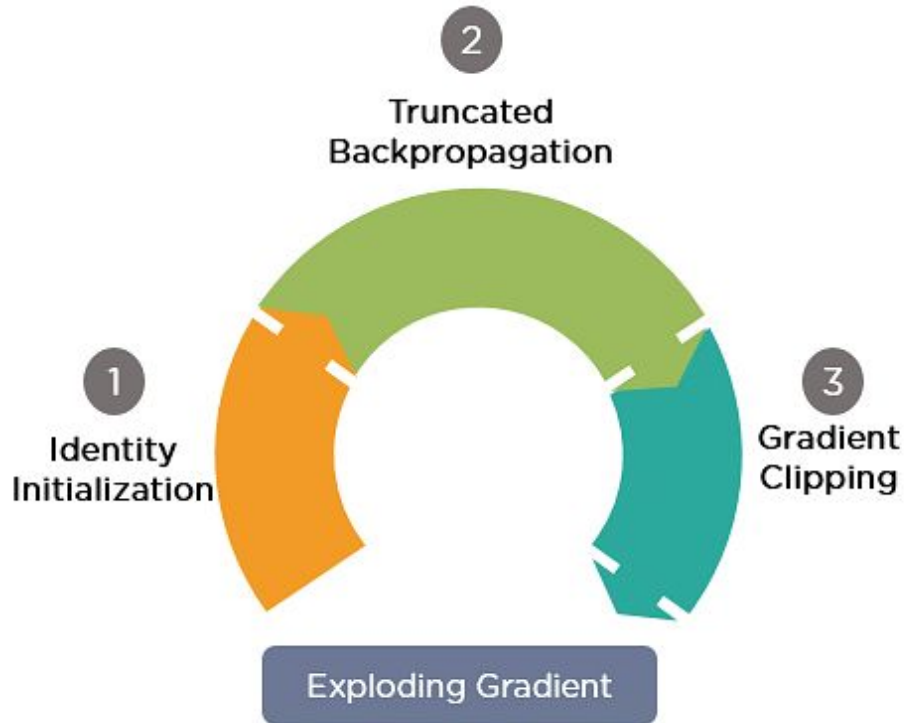
Vanishing Gradient Problem



Exploding Gradient Problem

- While training a neural network, if the slope tends to grow exponentially instead of decaying, this is called an **Exploding Gradient**.
- This problem arises when large error gradients accumulate, resulting in very large updates to the neural network model weights during the training process.
- Long training time, poor performance, and bad accuracy are the major issues in gradient problems.

Gradient Problem Solutions



- Now, let's discuss the most popular and efficient way to deal with gradient problems, i.e., Long Short-Term Memory Network (LSTMs).
- First, let's understand Long-Term Dependencies.
- Suppose you want to predict the last word in the text: "The clouds are in the _____."
- The most obvious answer to this is the "sky." We do not need any further context to predict the last word in the above sentence.
- Consider this sentence: "I have been staying in Spain for the last 10 years...I can speak fluent _____."
- The word you predict will depend on the previous few words in context.
- Here, you need the context of Spain to predict the last word in the text, and the most suitable answer to this sentence is "Spanish."
- The gap between the relevant information and the point where it's needed may have become very large. LSTMs help you solve this problem.

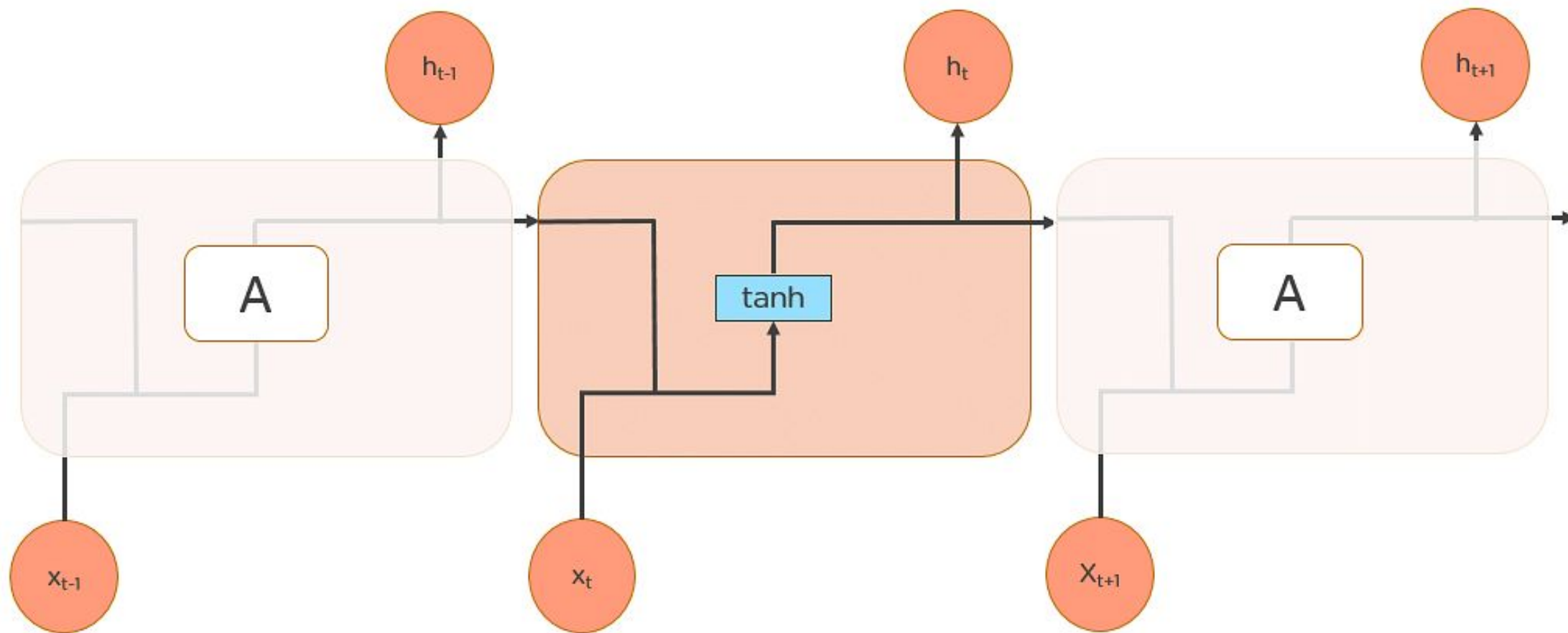
Backpropagation Through Time

- Backpropagation through time is when we apply a Backpropagation algorithm to a Recurrent Neural network that has time series data as its input.
- In a typical RNN, one input is fed into the network at a time, and a single output is obtained.
- But in backpropagation, you use the current as well as the previous inputs as input.
- This is called a timestep and one timestep will consist of many time series data points entering the RNN simultaneously.
- Once the neural network has trained on a timeset and given you an output, that output is used to calculate and accumulate the errors.
- After this, the network is rolled back up and weights are recalculated and updated keeping the errors in mind.

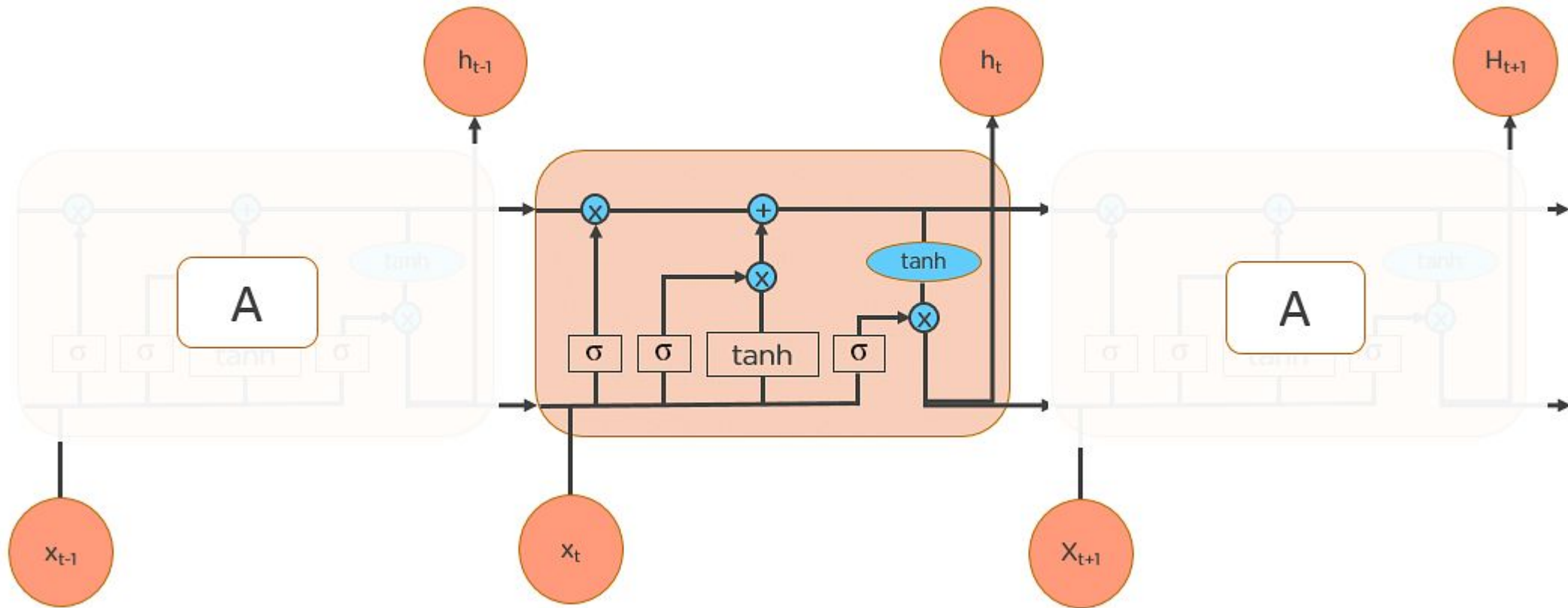
Long Short-Term Memory Networks

- LSTMs are a special kind of RNN — capable of learning long-term dependencies by remembering information for long periods is the default behavior.
- All RNN are in the form of a chain of repeating modules of a neural network.
- In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer.

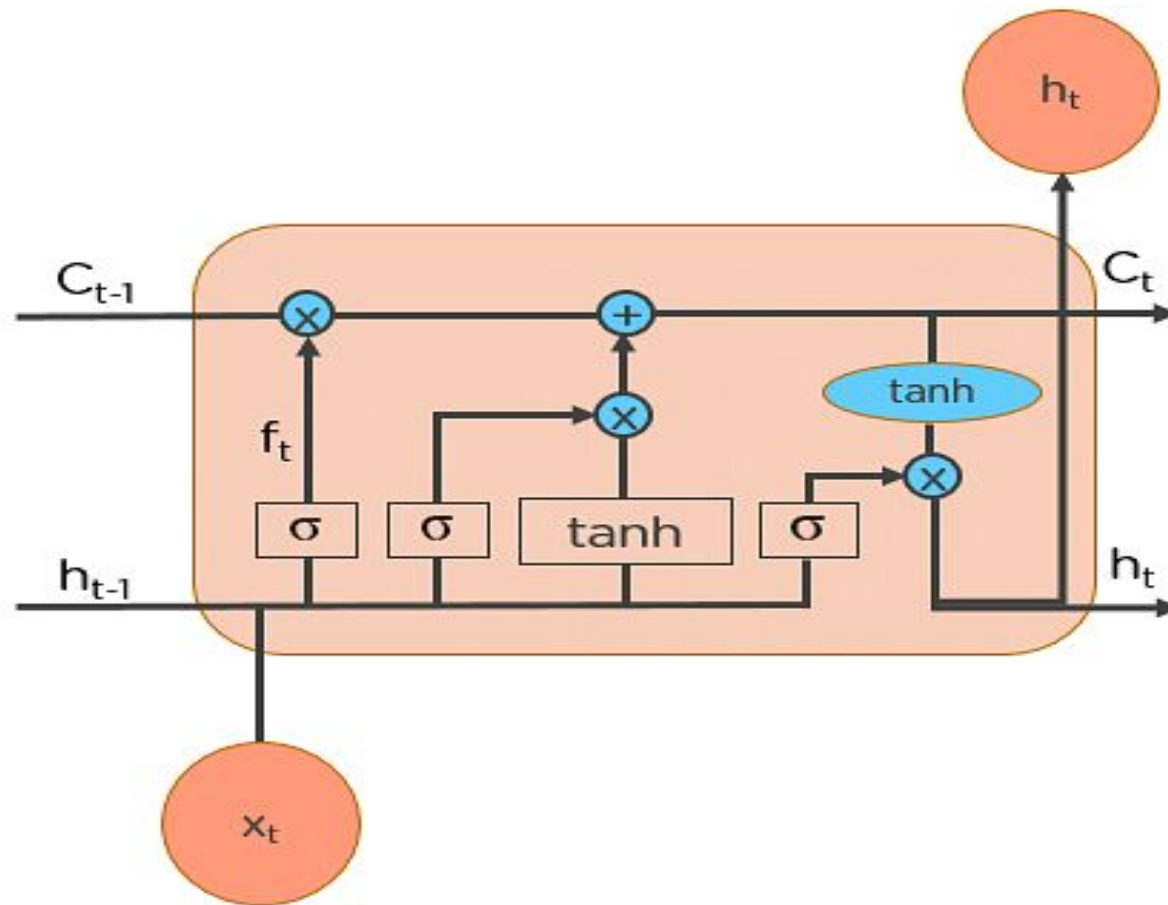
Long Short-Term Memory Networks



- LSTMs also have a chain-like structure, but the repeating module is a bit different structure.
- Instead of having a single neural network layer, four interacting layers are communicating extraordinarily.



Workings of LSTMs in RNN



Step 2: Decide How Much This Unit Adds to the Current State

- In the second layer, there are two parts. One is the sigmoid function, and the other is the tanh function.
- In the sigmoid function, it decides which values to let through (0 or 1).
- Tanh function gives weightage to the values which are passed, deciding their level of importance (-1 to 1).

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

i_t = input gate
Determines which information to let through based on its significance in the current time step

With the current input at $x(t)$, the input gate analyzes the important information — John plays football, and the fact that he was the captain of his college team is important. “He told me yesterday over the phone” is less important; hence it's forgotten. This process of adding some new information can be done via the input gate.

Step 3: Decide What Part of the Current Cell State Makes It to the Output

- The third step is to decide what the output will be.
- First, we run a sigmoid layer, which decides what parts of the cell state make it to the output.
- Then, we put the cell state through tanh to push the values to be between -1 and 1 and multiply it by the output of the sigmoid gate.

$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

o_t = output gate
Allows the passed in information to
impact the output in the current
time step

- Let's consider this example to predict the next word in the sentence:
- “John played tremendously well against the opponent and won for his team.
- For his contributions, brave ____ was awarded player of the match.”
- There could be many choices for the empty space.
- The current input brave is an adjective, and adjectives describe a noun. So, “John” could be the best output after brave.

	ANN	CNN	RNN
Basics	One of the simplest types of neural networks.	One of the most popular types of neural networks.	The most advanced and complex neural network.
Structural Layout	Its simplicity comes from its feed forward nature — information flows in one direction only.	Its structure is based on multiple layers of nodes including one or more convolutional layers.	Information flows in different directions, which gives it its memory and self-learning features.
Data Type	Fed on tabular and text data.	Relies on image data.	Trained with sequence data.
Complexity	Simple in contrast with the other two models.	Considered more powerful than the other two.	Fewer features than CNN but powerful due to its self-learning & memory potential.
Commendable Feature	Ability to work with incomplete knowledge and high fault tolerance.	Accuracy in recognizing images.	Memory and self-learning.
Feature type: spatial recognition	No	Yes	No
Feature type: Recurrent connections	No	No	Yes
Main Drawback	Hardware dependence.	Large training data required.	Slow and complex training and gradient concerns.
Uses	Complex problem solving such as predictive analysis.	Computer vision including image recognition	Natural language processing including sentiment analysis and speed recognition.



	ANN	CNN	RNN
Type of Data	Tabular Data, Text Data	Image Data	Sequence data
Parameter Sharing	No	Yes	Yes
Fixed Length input	Yes	Yes	No
Recurrent Connections	No	No	Yes
Vanishing and Exploding Gradient	Yes	Yes	Yes
Spatial Relationship	No	Yes	No
Performance	ANN is considered to be less powerful than CNN, RNN.	CNN is considered to be more powerful than ANN, RNN.	RNN includes less feature compatibility when compared to CNN.
Application	Facial recognition and Computer vision.	Facial recognition, text digitization and Natural language processing.	Text-to-speech conversions.
Main advantages	Having fault tolerance, Ability to work with incomplete knowledge.	High accuracy in image recognition problems, Weight sharing.	Remembers each and every information, Time series prediction.
Disadvantages	Hardware dependence, Unexplained behavior of the network.	Large training data needed, don't encode the position and orientation of object.	Gradient vanishing, exploding gradient.